Software Architecture Document

Collaborative Study Material and Online Quiz Platform

Team Members:
Sanjith K S (PES1UG24CS613)
Fathima Zahra (PES1UG24CS664)
Dhanush S Shekhar (PES1UG24CS662)

Version 1.0 | 19 October 2026

1. Introduction

1.1 Purpose

This document describes the software architecture of the Collaborative Study Material and Online Quiz Platform. It shows the system from several views (use-case, logical, process, deployment and data) so that every team member builds, tests and changes the system in line with one agreed design. It is the architectural companion to the SRS, the Use Case Flow, the Test Plan and the Security Requirements Specification.

1.2 Scope

The document covers the whole platform: the web user interface, the REST API server and its eleven feature modules, the PostgreSQL database, the external LLM service, and the build, test and deployment pipeline. It covers all functional requirements FR-01 to FR-14 and non-functional requirements NFR-01 to NFR-07. Components that are designed but scheduled for Sprint 2 are shown with a dashed border in every diagram, so the document describes both what is built today and the target architecture for 30 October 2026.

1.3 Definitions, Acronyms and Abbreviations

Term	Meaning
API	Application Programming Interface. Here: the REST/JSON interface under /api/v1.
Module	A Python package under app/modules/ that owns its database tables and exposes one public interface (service.py).
Service interface	The functions in a module's service.py. Other modules may call only these, never another module's tables.
Capability	A single permitted action such as material:upload or user:manage. Routes are guarded by capability, not by role name.
ORM	Object Relational Mapper (SQLAlchemy 2), which maps Python classes to database tables.
Migration	A versioned, reversible database schema change managed by Alembic.
LLM	Large Language Model, an external AI service used for question generation, answer evaluation and explanations.
CI	Continuous Integration: GitHub Actions runs lint and tests on every pull request; the lint-and-test check must pass before merge.

Term	Meaning
JWT / HS256	JSON Web Token signed with HMAC-SHA256, used as the access token.
FR / NFR	Functional / Non-Functional Requirement in the SRS.

1.4 References

- SRS Document (Documents/SRS Document.docx): FR-01 to FR-14, NFR-01 to NFR-07.
- Use Case Flow (Documents/UseCaseFlow.docx) and Use Case Diagram (Documents/UseCaseDiagram.drawio): UC-01 to UC-16.
- Test Plan Document (Documents/Test Plan Document.docx): unit and integration test cases.
- Security Requirements Specification (Documents/Security_Requirements_Document.pdf).
- Repository documentation: README.md, docs/architecture.md, docs/api/openapi-v1.json (API contract), docs/test-execution-log.md.
- UE24CS341A Software Engineering, Unit 2 slides: Software Architecture, architectural patterns and documentation guidelines.

2. Architectural Representation

The architecture is described through the views listed below. Following the course guidelines, each diagram fits on one page, decomposes the system hierarchically, and carries its own legend that defines what the boxes and the lines mean.

View	What it shows	Notation	Section
Context (client-server)	Users, the system boundary and the external systems	Boxes and arrows	2.1
Use-case view	Architecturally significant use cases and the modules that realise them	Table	4
Logical view	Layers and the eleven feature modules with their allowed dependencies	Layer diagram, package diagram	5.1
Process view	How components collaborate at run time for key scenarios	UML sequence diagrams	5.2
Data view	Database tables, their owning module and relationships	Entity relationship diagram	5.3
Deployment view	Machines, containers, storage and the CI/CD pipeline	Deployment diagram	6

Architectural style. The platform combines three patterns from the course: Client-Server (browsers send requests, the server holds business logic, security and data), Layered (presentation, API, service, data access, infrastructure) and a modular monolith inside the server: one deployable process split into independent feature modules with enforced boundaries. A modular monolith gives the loose coupling and high cohesion that NFR-07 asks for, without the operational cost of running many separate services for a three-person team in two sprints. Because modules talk only through service interfaces, any module can later be split into its own service if load requires it (NFR-05).

2.1 System Context

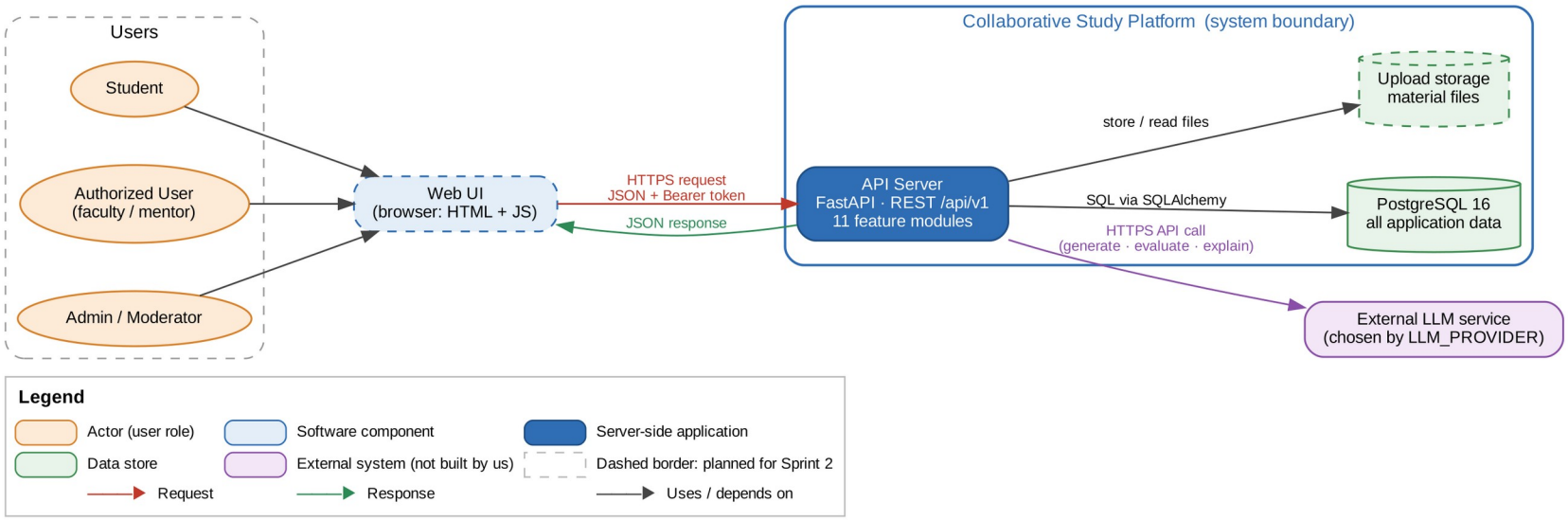


Figure 1: System context: client-server view of the platform and its external systems

Three kinds of users reach the platform through the web UI in a browser. The browser sends HTTPS requests with a JSON body and a bearer token to the API server, which is the only component that touches the database, the upload storage and the external LLM service. The LLM provider is configurable (LLM_PROVIDER) and can be switched off entirely; the rest of the platform keeps working when it is unavailable.

3. Architectural Goals and Constraints

3.1 Goals

ID	Goal	Source	How the architecture meets it
G-1	Respond quickly: pages and search within 2 s, quiz scoring within 3 s (95th percentile, 50 users)	NFR-01	Stateless API process, indexed columns for search filters, objective scoring is pure in-process code with no LLM call; LLM work is isolated in its own module.
G-2	Protect credentials and enforce role-based access on the server	NFR-02, SEC	scrypt password hashing, signed expiring tokens, capability guards on every protected route, user re-read on every request, secrets only in environment variables.
G-3	Simple, discoverable interface with visual analytics	NFR-03	Dedicated presentation layer consuming one documented API; analytics module returns ready-to-chart data.

ID	Goal	Source	How the architecture meets it
G-4	Never lose committed attempts or uploads	NFR-04	PostgreSQL transactions, attempt answers and score written in one transaction, persistent Docker volumes, reversible migrations.
G-5	Grow with users, content and new domains	NFR-05	Stateless API can run as several replicas; modules can be extracted into services; new subjects are data, not code.
G-6	Trustworthy AI content and scoring	NFR-06	LLM questions are saved as drafts until a person approves them; objective scoring is deterministic and never uses the LLM; every LLM call is recorded.
G-7	Change one module without breaking others	NFR-07	Each module owns its tables and exposes one service interface; an automated boundary test fails the build if a module imports another module's internals.

3.2 Constraints

- **Team and time:** three developers, two one-week sprints (19 to 30 October 2026); the design must allow parallel work on separate modules.
- **Technology:** Python 3.12, FastAPI, SQLAlchemy 2, Alembic, PostgreSQL 16 (SQLite allowed for local development), Docker for deployment.
- **Process:** every story is one pull request; main is protected and accepts a merge only after the lint-and-test CI check passes.
- **Security:** no secret may be committed; all secrets come from environment variables (see the Security Requirements Specification).
- **External dependency:** the LLM is a third-party service with variable latency and availability; the platform must degrade gracefully without it.

4. Use-Case View

4.1 Architecturally Significant Use Cases

The use cases below were selected because each one exercises a different architectural mechanism. Together they touch every module and every layer.

Use case	Why it is architecturally significant	Modules involved	FR
UC-01 Register & Login	Drives authentication, token handling and the capability model used by every other route	accounts	FR-01
UC-06 Create / Join Study Room (channel)	Membership decides access to materials, quizzes and chat	channels, accounts	FR-02, FR-03
UC-03 / UC-04 Upload, search and download material	File storage, search indexes and content status	materials, channels	FR-04
UC-07 / UC-08 Question bank and create quiz	Cross-module call from quizzes to the question bank through its service interface	questions, quizzes	FR-05
UC-09 Attempt Quiz	Performance-critical path (NFR-01) and data that must never be lost (NFR-04)	quizzes, questions	FR-06

Use case	Why it is architecturally significant	Modules involved	FR
UC-10 / UC-11 AI quiz generation and descriptive evaluation	External dependency with a human review gate and failure handling	llm, questions, quizzes	FR-07, FR-08, FR-09
Channel and direct chat	High message volume; must stay isolated from quiz scoring	chat, channels	FR-10, FR-11
UC-12 / UC-13 Performance analysis and recommendations	Read-only module that derives data from quiz attempts	analytics, quizzes, materials	FR-12
UC-14 / UC-15 Reputation and moderation	Polymorphic reports across all content types	reputation, moderation	FR-13
UC-16 Manage users & channels	Admin-only capabilities, suspension takes effect immediately	accounts, channels, moderation, notifications	FR-14

5. Logical View

5.1 Architecture Overview: Package and Subsystem Layering

The server is organised in five layers. A request enters through the API layer, which validates it and checks the caller's capability, then calls a feature module's service. The service applies the business rules and reads or writes only its own tables through the data access layer.

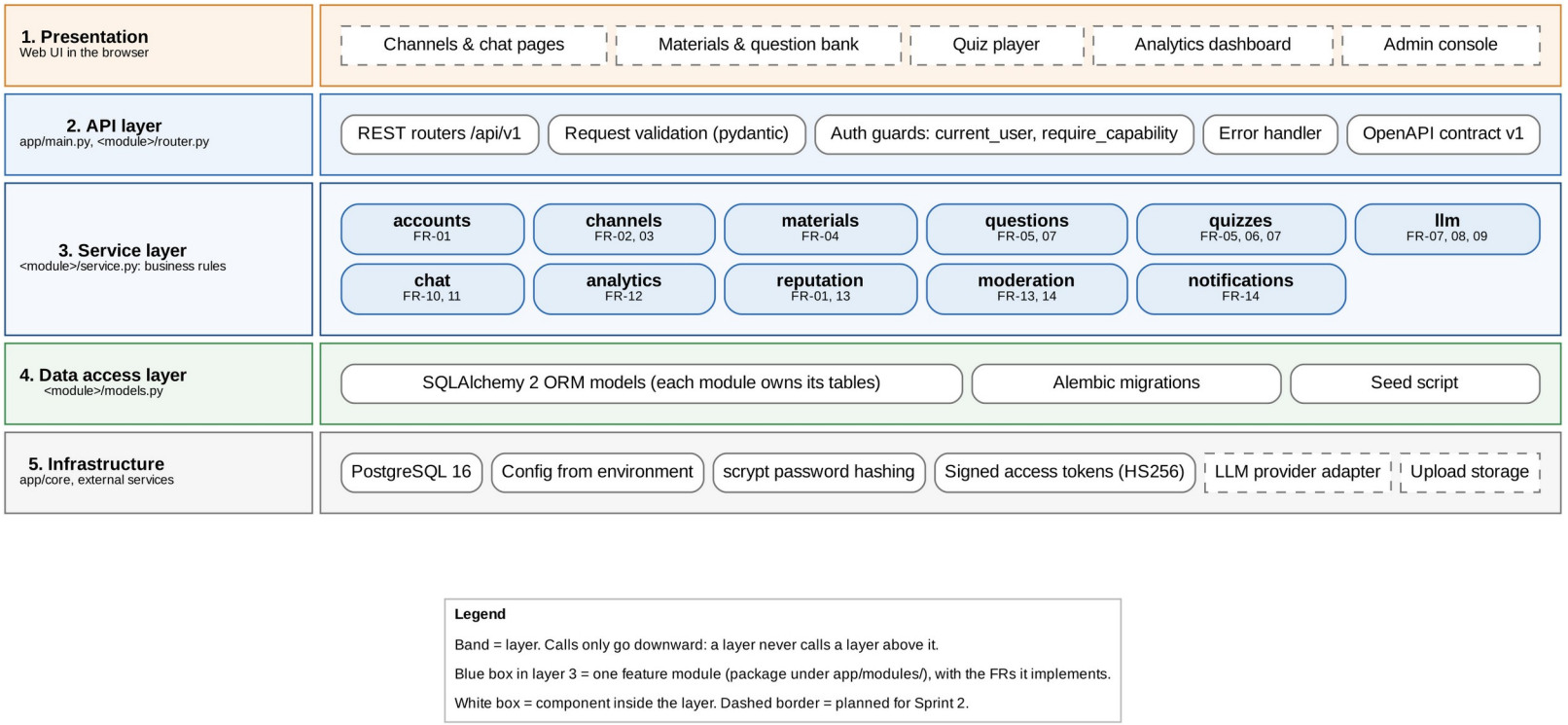


Figure 2: Layered view: five layers and the eleven feature modules

The service layer is split into eleven feature modules. Figure 3 shows which module may call which. The rule is enforced by an automated test (tests/test_module_boundaries.py) that runs in CI on every pull request.

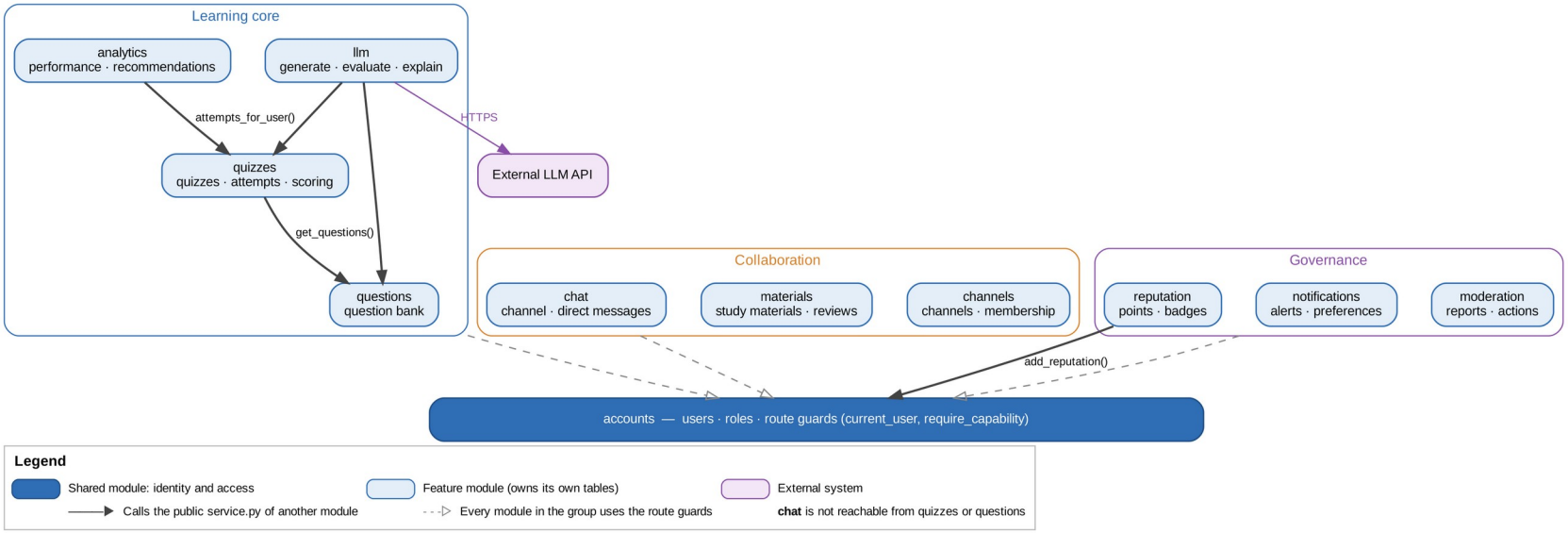


Figure 3: Module dependency view: feature modules, groups and allowed calls

Module	Responsibility	Owns tables	May call
accounts	Registration, login, profiles, roles, route guards	users	none
channels	Normal and authorized channels, approval, membership	channels, channel_memberships	accounts (guards)
materials	Upload, search, download, rating, review of materials	study_materials, material_reviews	accounts
questions	Question bank: contribute, search, rate; LLM drafts	questions, question_ratings	accounts
quizzes	Build and publish quizzes, attempts, deterministic scoring	quizzes, quiz_questions, quiz_attempts, attempt_answers	questions, accounts
llm	Question generation, adaptive difficulty, descriptive evaluation, explanations	llm_requests	questions, quizzes, accounts
chat	Channel threads, direct conversations, blocking, sharing links	channel_messages, conversations, conversation_participants, direct_messages, user_blocks	accounts
analytics	Performance trends, weak topics, recommendations	none (derived)	quizzes, accounts

Module	Responsibility	Owns tables	May call
reputation	Reputation points, badges, leaderboard	reputation_events, badges, user_badges	accounts
moderation	Reports, moderator and admin actions	reports, moderation_actions	accounts
notifications	In-app notifications and preferences	notifications, notification_preferences	accounts

Cohesion and coupling. Each module groups the operations of one feature (high cohesion). Modules exchange plain data objects such as QuestionView and AttemptResult rather than database rows, so a change inside one module's tables cannot leak into another (loose coupling). Quiz scoring lives in quizzes/scoring.py as pure functions, and no module that quizzes depends on can reach chat, so a change to chat can never require a change to scoring.

5.2 Process View

5.2.1 Processes

At run time the system consists of the processes below. The API server is stateless: all state is in PostgreSQL or the upload storage, so several API processes can run side by side behind a load balancer.

Process	Runs on	Responsibility
Web browser	User's device	Renders the UI and calls the REST API with the user's bearer token.
uvicorn / FastAPI (api)	api container	Handles every HTTP request; hosts all eleven modules in one process.
Migration step (alembic upgrade head)	api container, at start-up	Brings the database schema to the latest version before the API accepts traffic.
PostgreSQL server	db container	Stores all application data with transactions and constraints.
External LLM service	Third party	Generates and evaluates content on request.
CI runner	GitHub Actions	Runs ruff and pytest (with a PostgreSQL 16 service) on every pull request.

5.2.2 Process to Design Elements

The sequence diagrams below show how the processes and modules collaborate in three key scenarios. Solid arrows are calls or requests, dashed arrows are returns, numbered circles give the order, and alt and loop frames show alternatives and repetition. Notes state the rule the scenario protects. Scenarios 2 and 3 show the target design for stories scheduled in the sprints.

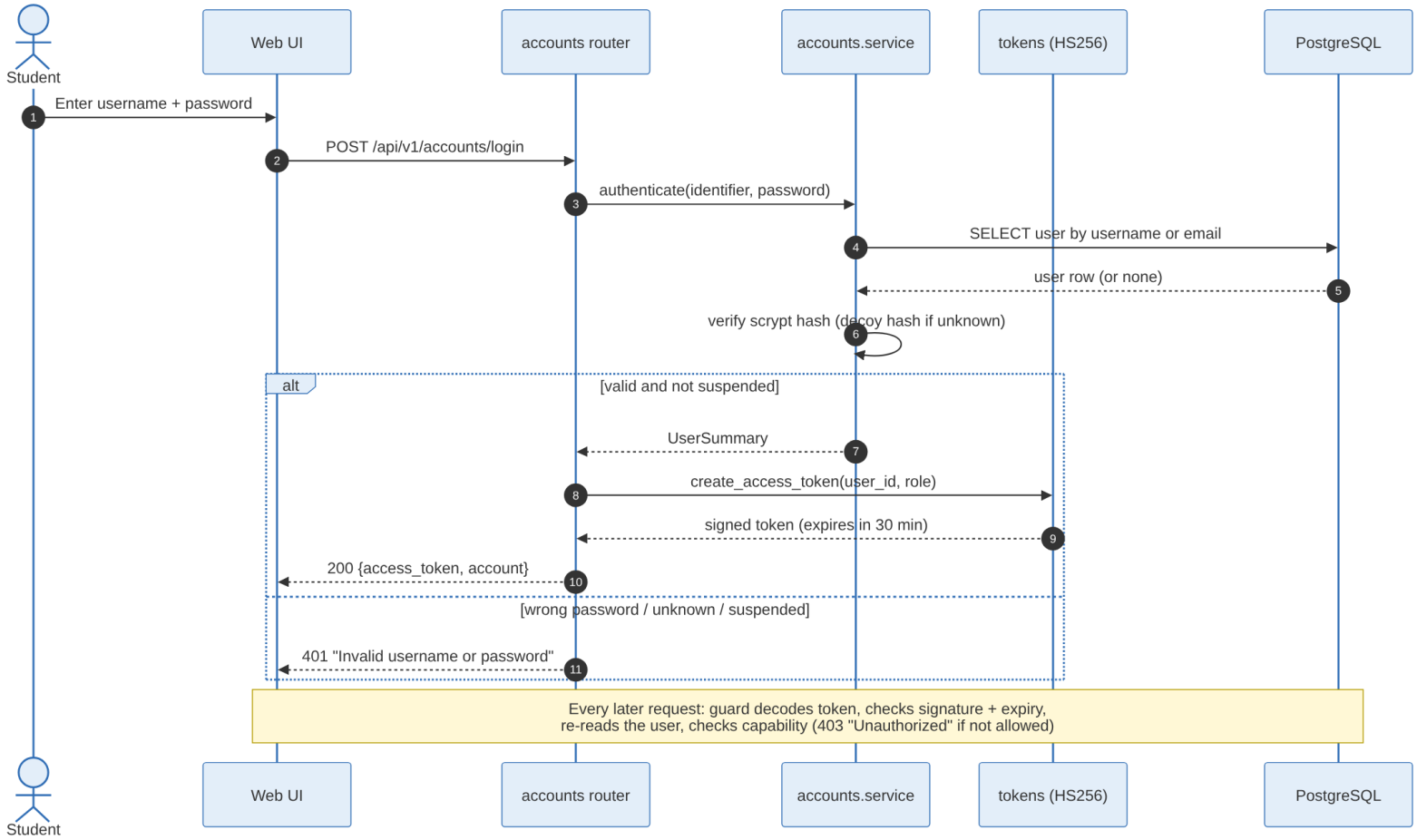


Figure 4: Sequence 1: log in and access a protected feature (UC-01, FR-01, NFR-02)

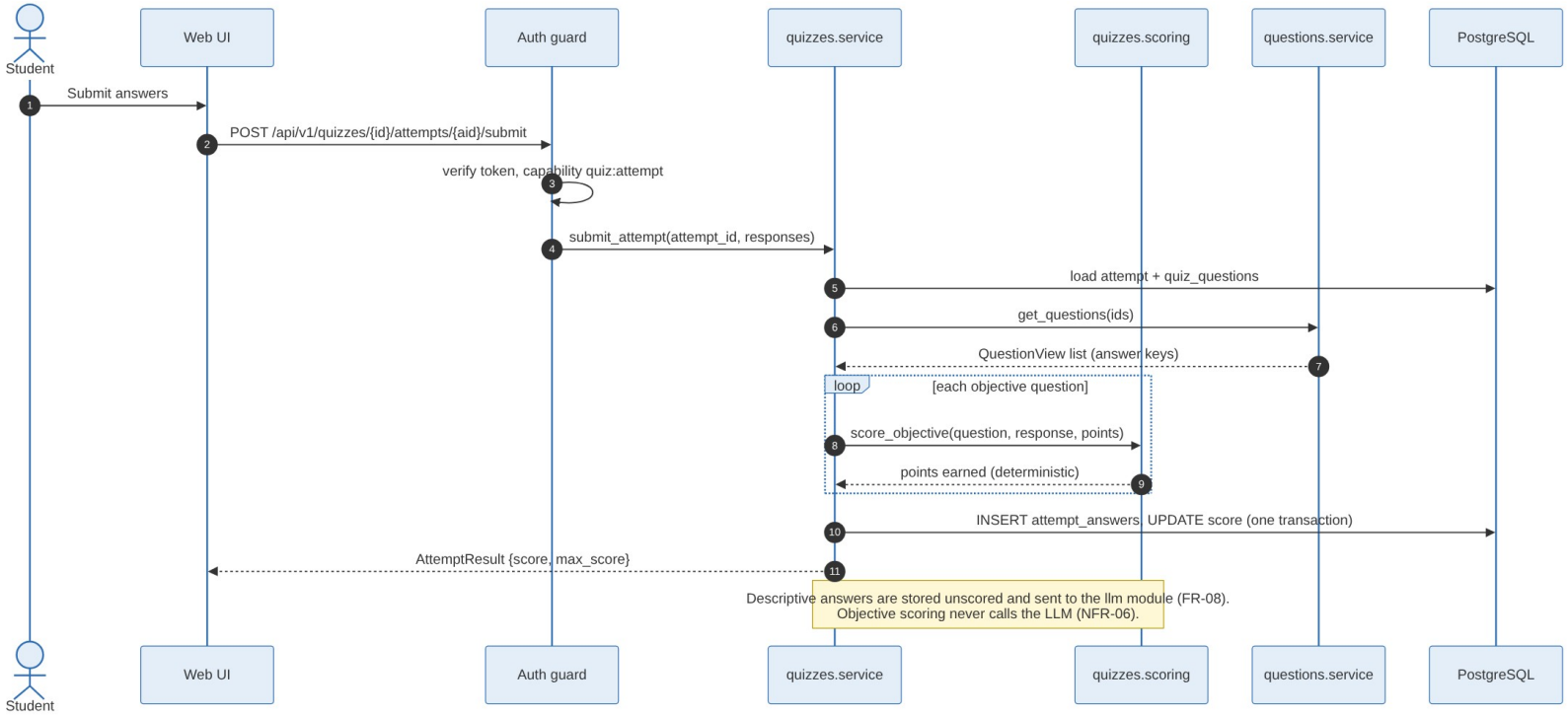


Figure 5: Sequence 2: submit a quiz attempt and score it deterministically (UC-09, FR-06, NFR-01, NFR-04)

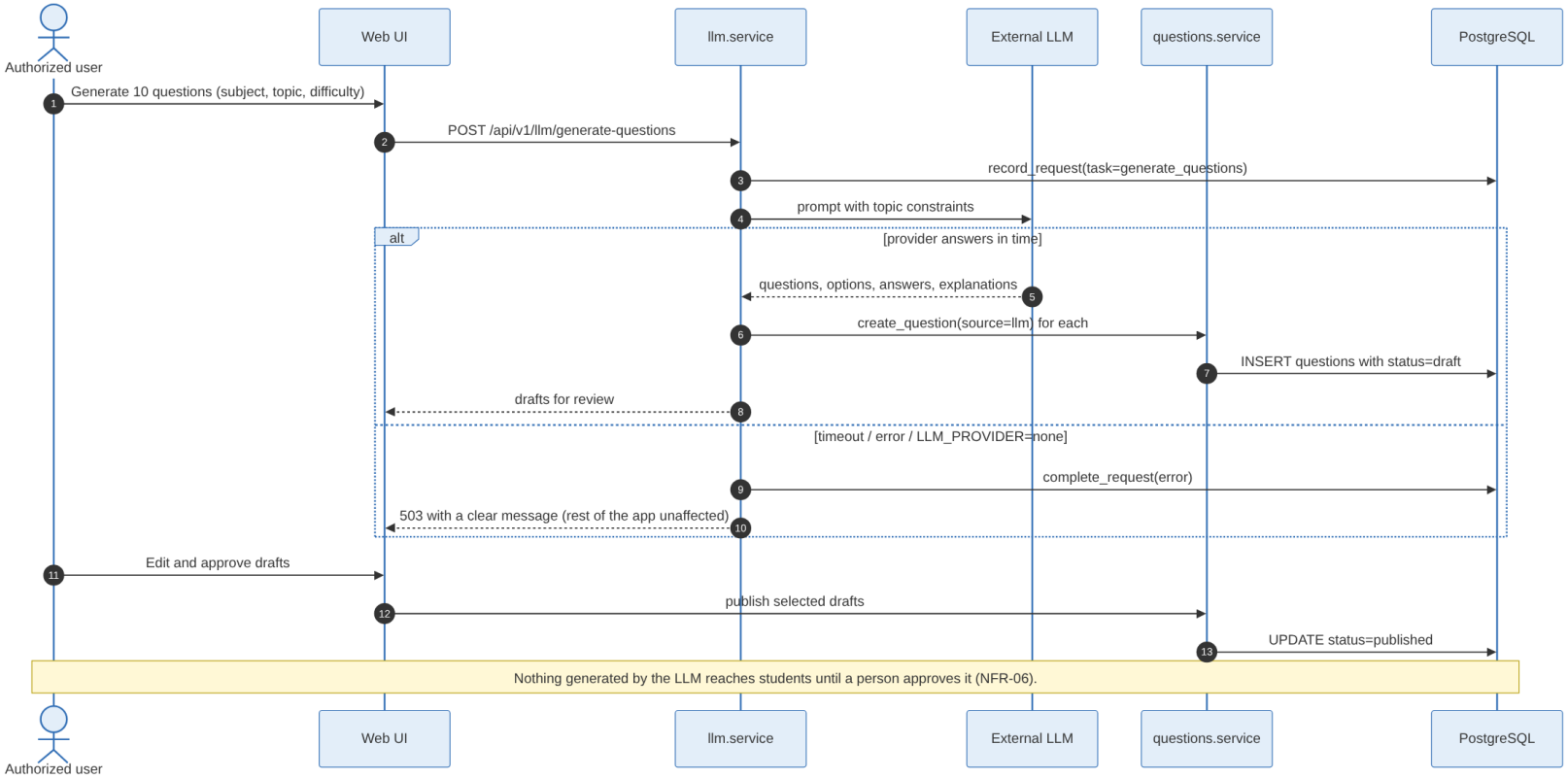


Figure 6: Sequence 3: generate questions with the LLM behind a human review gate (UC-10, FR-07, NFR-06)

5.2.3 Model Dependencies

- quizzes depends on questions (read question views and answer keys).
- llm depends on questions (store drafts) and quizzes (adaptive difficulty during an attempt).
- analytics depends on quizzes (attempt results); it stores nothing of its own.
- reputation depends on accounts (cached reputation total).
- Every module with routes depends on accounts for the route guards. No module depends on chat.

5.2.4 Processes to the Implementation

Element	Implementation
Application factory and router registration	app/main.py
Configuration, database session, password hashing	app/core/config.py, app/core/database.py, app/core/security.py

Element	Implementation
Feature module	app/modules/<module>/models.py (tables), service.py (public interface), router.py (HTTP routes)
Allowed dependencies between modules	app/modules/__init__.py (ALLOWED_DEPENDENCIES), enforced by tests/test_module_boundaries.py
Schema versions	alembic/versions/0001_initial_schema.py, 0002_add_username_to_users.py, ...
Sample data per role	app/seed.py (python -m app.seed)
Published API contract	docs/api/openapi-v1.json, checked against the code in CI
Build and deployment	Dockerfile, docker-compose.yml, scripts/deploy.sh, .github/workflows/ci.yml

5.3 Data View

All data lives in one PostgreSQL database, but every table belongs to exactly one module (shown in brackets). Foreign keys keep the data consistent across modules without any Python imports between them. Links that can point at any kind of content, such as a report target or a resource shared in chat, are stored as a type plus an id.

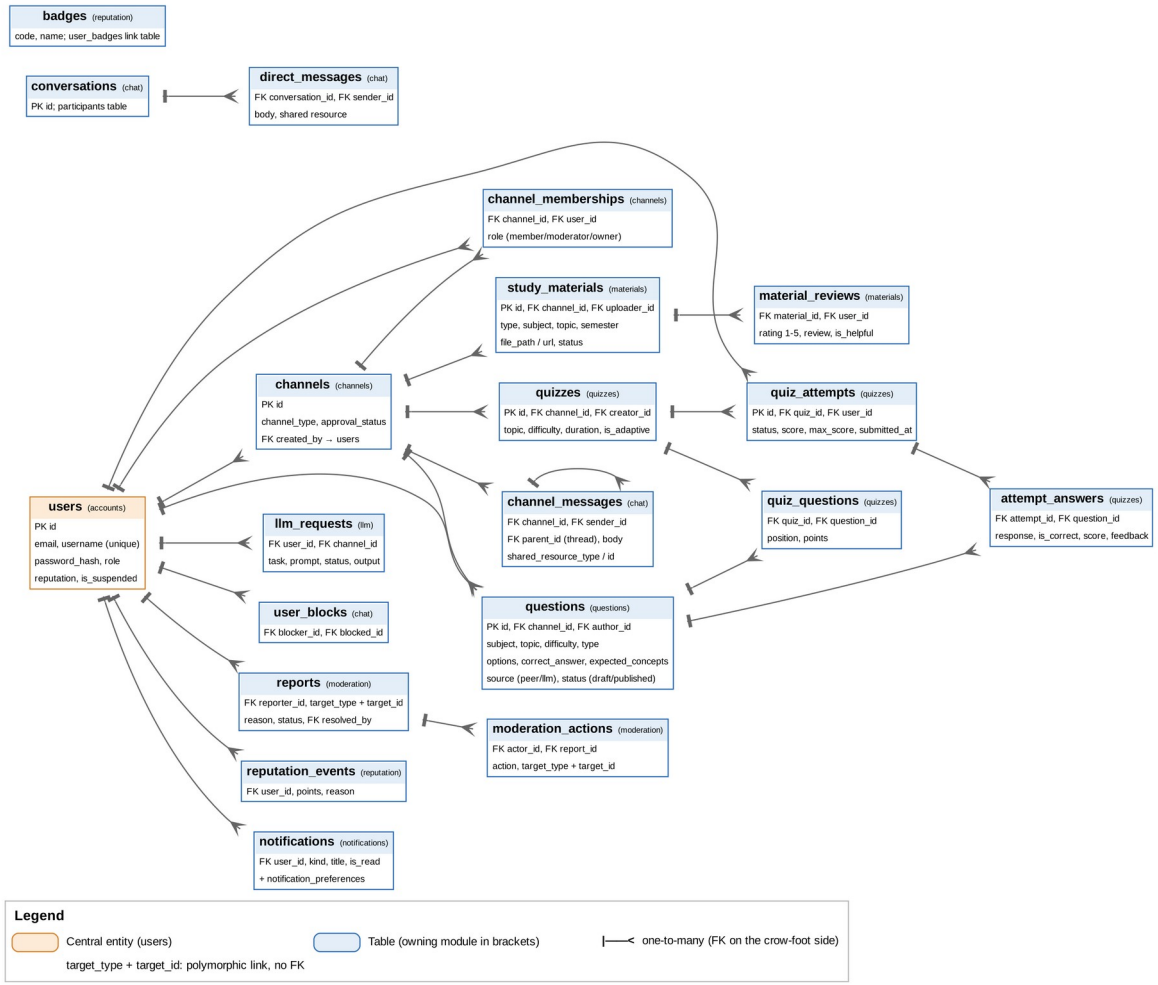


Figure 7: Data view: tables, owning modules and relationships

6. Deployment View

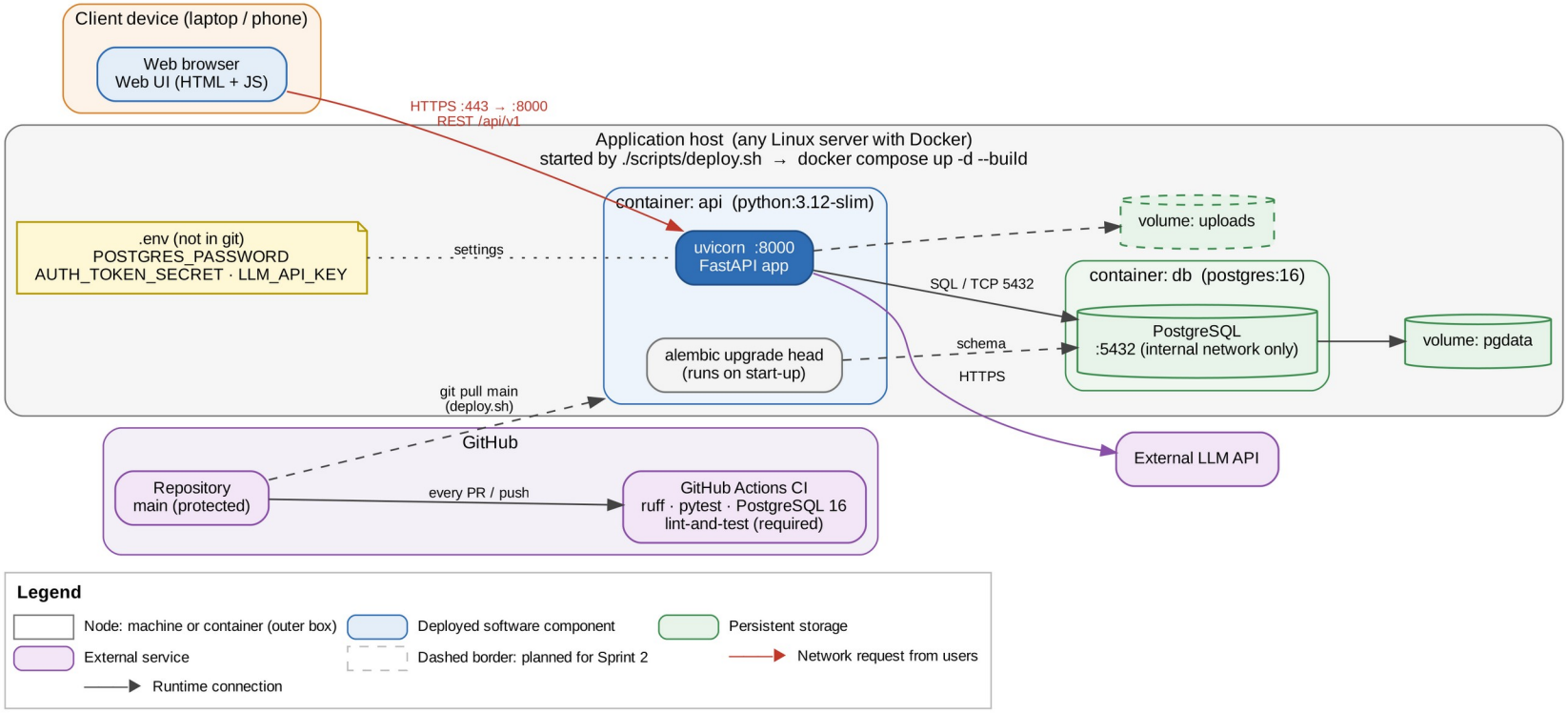


Figure 8: Deployment view: client, application host, containers, storage and CI/CD

Node	Description
6.1 Client device	Any modern browser on a laptop, desktop or phone. Nothing is installed; the web UI is served by the platform.
6.2 Application host	One Linux server with Docker. ./scripts/deploy.sh pulls main and runs docker compose up -d --build. Secrets are read from a .env file on the host that is never committed.
6.3 api container	python:3.12-slim image. Runs database migrations on start-up, then uvicorn serving the FastAPI app on port 8000.
6.4 db container	postgres:16 on the internal Docker network only (not exposed to the internet). Data is kept on the pgdata volume, so it survives restarts and redeploys.
6.5 Upload storage	A Docker volume for uploaded material files (Sprint 2); can be moved to object storage without changing other modules.

Node	Description
6.6 GitHub	Hosts the repository. GitHub Actions runs ruff and pytest against PostgreSQL 16 on every pull request; main accepts merges only when lint-and-test passes.
6.7 External LLM API	Reached over HTTPS from the llm module only. The key is LLM_API_KEY in the environment.

7. Performance

NFR-01 sets the targets: page loads and search or filter within 2 seconds, and quiz submission with objective scoring within 3 seconds, at the 95th percentile with up to 50 concurrent users. LLM operations are excluded from these targets but must show progress and fail cleanly.

Concern	Architectural tactic
Search and filter speed	Indexes on subject, topic, channel and owner columns; queries filter in the database, not in Python.
Quiz scoring time	Scoring is pure in-process code over data loaded in two queries (attempt questions, question views); no network calls.
Slow or failing LLM	Isolated in the llm module with request records and a clear error; the rest of the API is unaffected.
Growth in users	The API process is stateless (token-based auth, no server sessions), so more replicas can be added; PostgreSQL handles concurrency.
Measuring	Integration tests exercise the critical paths in CI; response times are checked against NFR-01 during system testing.

8. Quality

Quality attribute	NFR	How the architecture supports it	Evidence
Security	NFR-02	Hashed passwords, signed expiring tokens, capability guards on the server, immediate suspension, secrets only in the environment, protected main branch	tests/test_accounts.py, tests/test_repository_hygiene.py, Security Requirements Specification
Usability	NFR-03	Separate presentation layer over one consistent, documented API; analytics returns chart-ready data	Web UI stories (Sprint 2)
Reliability	NFR-04	Database transactions, constraints, persistent volumes, reversible migrations tested on SQLite and PostgreSQL	tests/test_migrations.py
Scalability	NFR-05	Stateless API, modules that can be extracted to services, data-driven subjects and channels	Module boundaries in section 5.1
AI trustworthiness	NFR-06	Draft-then-approve flow for LLM content, deterministic objective scoring, audit record of LLM requests, reporting of AI content	Sequence 3, tests/test_services.py
Maintainability	NFR-07	Modules own their data and expose one interface; dependency rules enforced in CI; versioned API contract; one PR per story	tests/test_module_boundaries.py, docs/api/openapi-v1.json